

3.0 TCP-IP Suite and Transport Layer

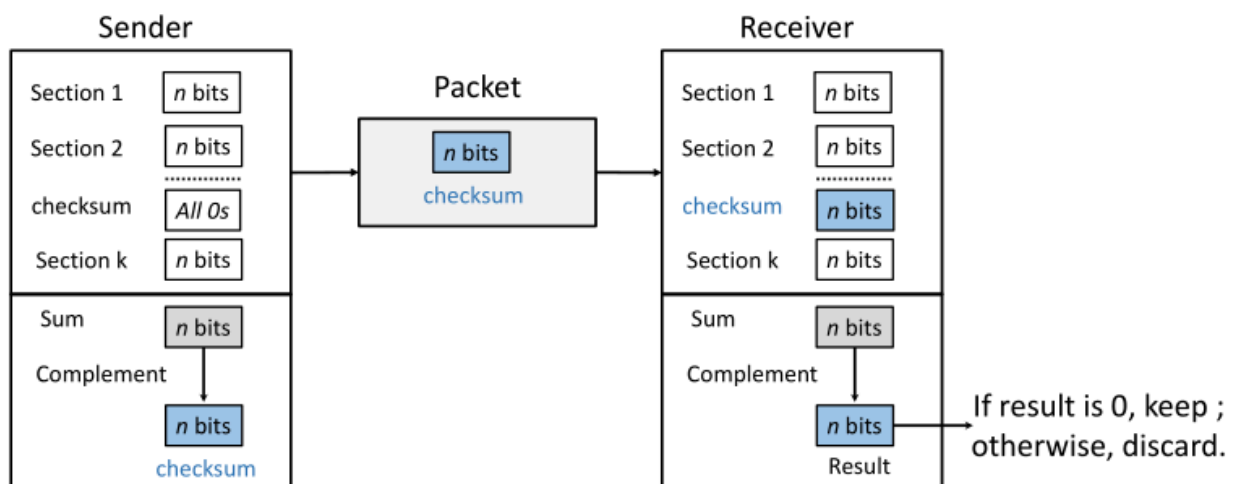
Learning Points:

- ☐ Learn and understand how checksum works at the sender and receiver side
- ☐ Understand how UDP provides connectionless, low-overhead data transmission
- ☐ Understand how TCP establishes, maintains and terminates a reliable connection

Checksum

Error method used by TCP/IP

- protects against corruption during transmission
- added to TCP header



Sender:

1. Packet is fragmented into n sections of k bits each
2. Packets are processed with ones complement arithmetic and complemented to create the checksum

Receiver:

1. Packet received in n sections of k bits each
2. Addition via ones complement arithmetic
3. summed to provided checksum and complemented

If result yields all bits are 0 Data is uncorrupted!

* **ones complement arithmetic** - a process where overflow from sum of each segment onto result is wrapped and used as next addition

$$\begin{array}{r}
 1111 \\
 + 0101 \\
 \hline
 \text{Carry } 10100 \\
 + 1 \\
 \hline
 \boxed{0101} \text{ Answer}
 \end{array}$$

* all higher order functions have their individual checksum fields covering the entire packet

- checksum in IP only covers changes in IP header, not the original data
 - checksum in TCP / UDP cover end to end integrity and involve checking actual data
- saves processing power, eg for router*

Transport Layer

Transport Protocols

- **User Datagram Protocol** - minimal overhead, unreliable communication
- **Transmission control protocol** - reliable, ordered, error checked delivery of data between applications over a network

UDP data

Minimal overhead, unreliable communication.

- connection less
- unreliable
- fast and efficient: small 8 byte header

3 sections - pseudo header + UDP header + data

- data is always padded to be 16 bits
 - * eg: video conference

TCP

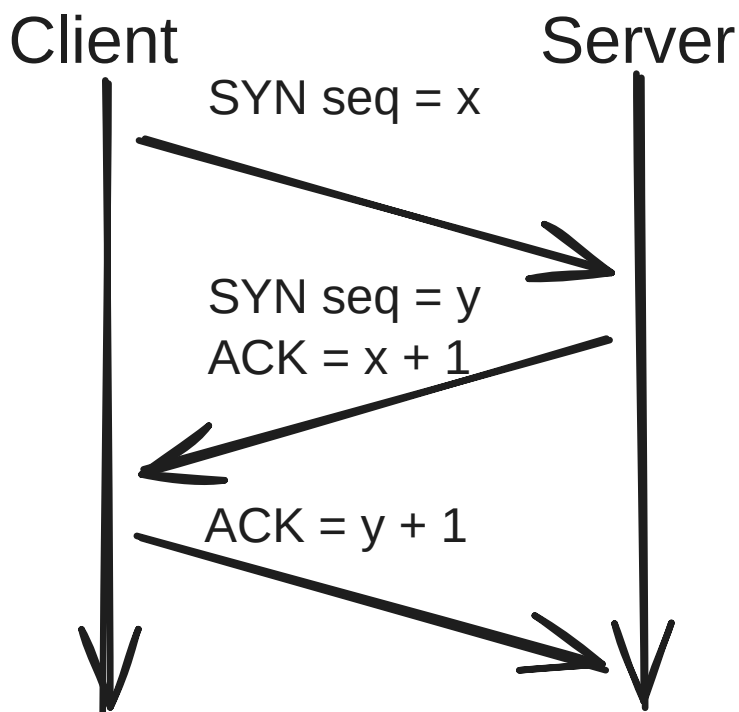
reliable error checked delivery of data between applications over a network

ACK - Next expected byte number

SYN - Seq number of first byte of data in TCP stream

Connection establishment - Three way handshake

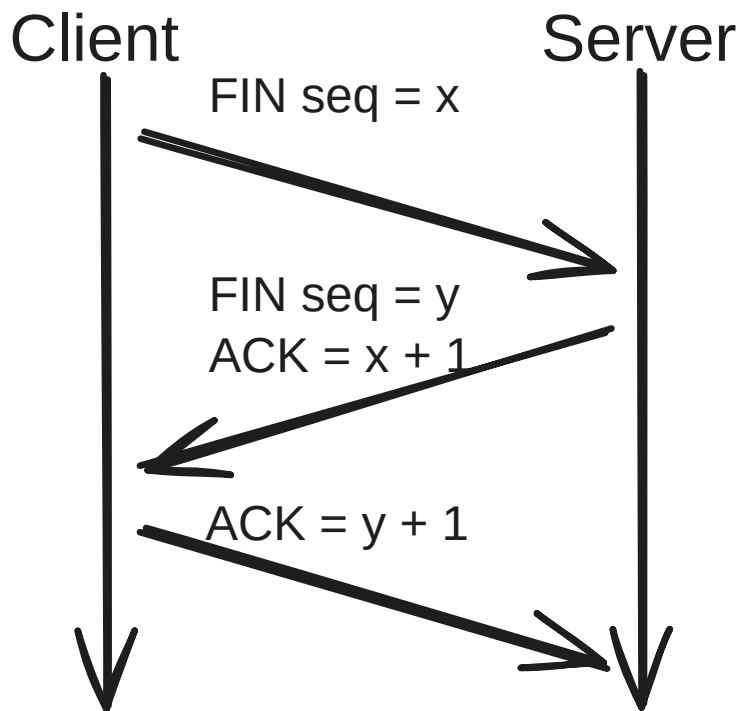
1. Client sends a SYN request to the server
2. Server acknowledges start of communication and replies with a SYN ACK
3. Client replies with ACK to acknowledge receipt



Connection termination - Three way Termination and Four way Half Close Teardown

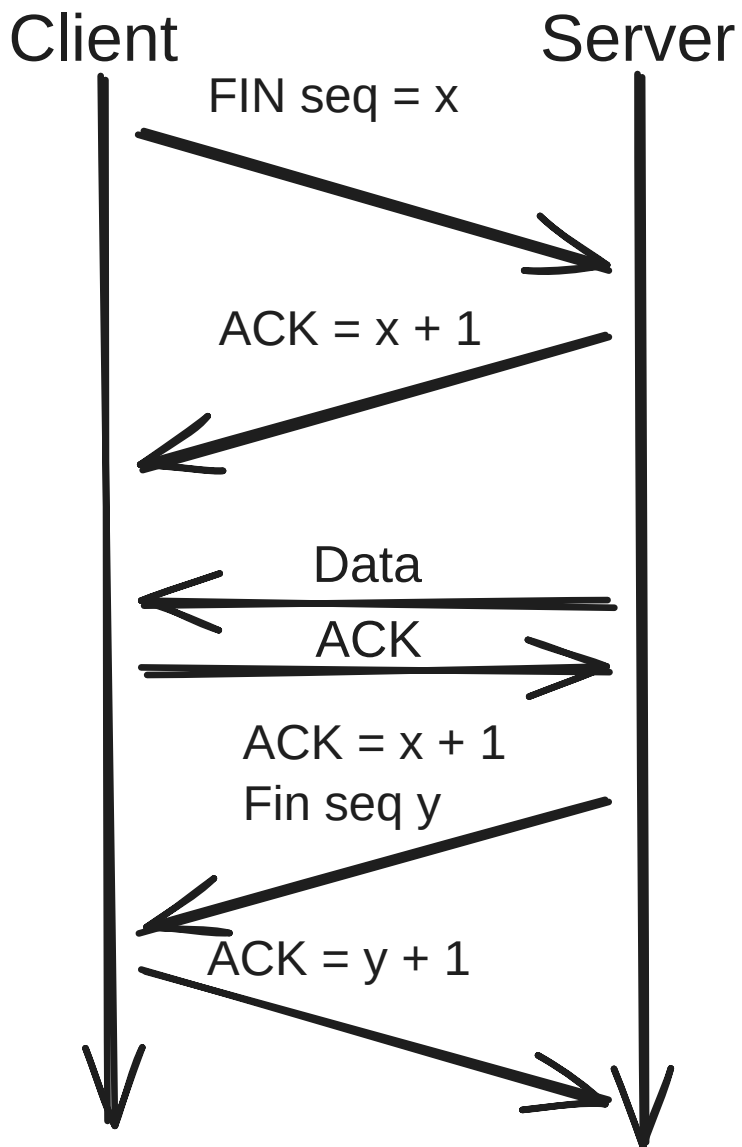
Three way Termination

1. Client sends FIN
2. Server confirms receipt of FIN and announces closure
3. Client replies with ACK



Four way Half Close Teardown - One end stops sending while still receiving data

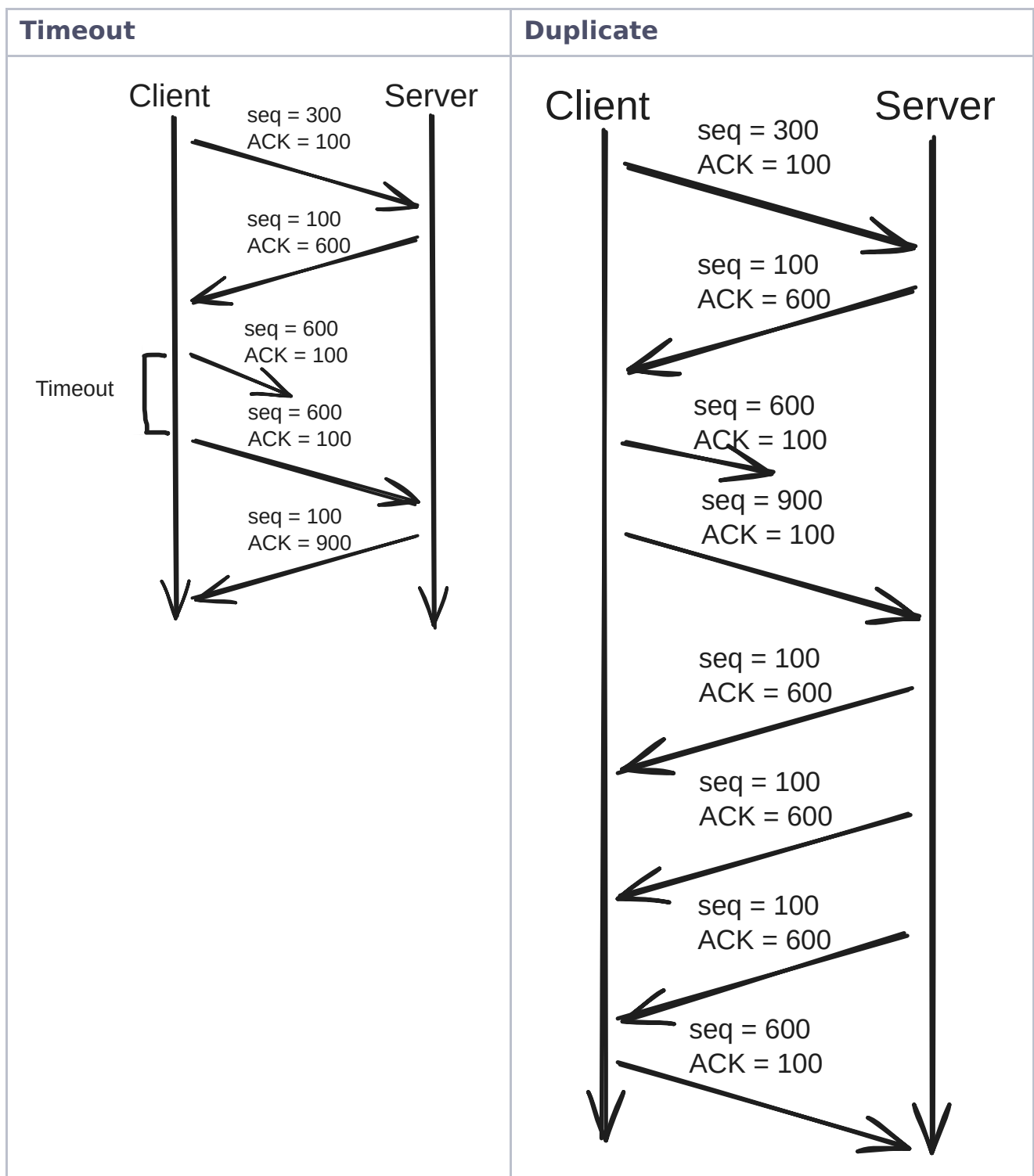
- both sides can issue the request
- second FIN not immediately generated



* sequence number: first byte's sequence number in any particular sequence

Re-transmission of lost segment

Timeout	Duplicate
retransmits upon timeout	re-transmits upon receiving three duplicate requests



Sliding window / flow control:

- improves network capacity usage

- send multiple packages before waiting for ACK

